DSA Patterns Pseudocode

1. Sliding Window

Use: Subarrays, substrings, max/min sum/length

```
FUNCTION slidingWindow(arr, k OR condition):
    windowStart = 0
    result = 0 or empty structure
    FOR windowEnd FROM 0 TO length(arr)-1:
        ADD arr[windowEnd] to currentWindow (sum, set, etc.)
        WHILE window size > k OR condition not met:
            REMOVE arr[windowStart] from currentWindow
            windowStart += 1
        UPDATE result if needed
    RETURN result
```

2. Two Pointers

Use: Pair sums, sorted arrays, palindrome check

```
FUNCTION twoPointers(arr, target):
    SORT arr if unsorted
    left = 0
    right = length(arr)-1
    WHILE left < right:
        sum = arr[left] + arr[right]
        IF sum == target:
            RETURN [left, right]
        ELSE IF sum < target:
            left += 1
        ELSE:
            right -= 1
    RETURN no solution
```

3. Hashing

Use: Count frequencies, complement sum, duplicates

```
FUNCTION useHash(arr, target):
    map = empty
    FOR each element in arr:
        IF complement = target - element exists in map:
            RETURN indices
        ELSE:
            map[element] = index
```

4. Binary Tree DFS

Use: Traversals, max/min depth, path problems

```
FUNCTION dfs(node):
    IF node == null:
        RETURN
    PROCESS node.value
    CALL dfs(node.left)
    CALL dfs(node.right)
```

DFS with return value (max depth example):

```
FUNCTION maxDepth(node):
    IF node == null:
        RETURN 0
    left = maxDepth(node.left)
    right = maxDepth(node.right)
    RETURN 1 + MAX(left, right)
```

5. Binary Tree BFS

Use: Level order traversal, shortest path, connected nodes

```
FUNCTION bfs(root):
    queue = [root]
    WHILE queue not empty:
        node = DEQUEUE
        PROCESS node.value
        FOR each child in node.children:
            ENQUEUE child
```

6. Backtracking

Use: Combinations, permutations, subset problems

```
FUNCTION backtrack(path, choices):
    IF base case satisfied:
        STORE path in results
        RETURN
    FOR each choice in choices:
        IF choice valid:
            ADD choice to path
            CALL backtrack(path, remaining choices)
            REMOVE choice from path (backtrack)
```

7. Fast & Slow Pointers

Use: Cycle detection, middle of linked list, palindrome linked list

```
FUNCTION detectCycle(head):
    slow = head
    fast = head
    WHILE fast != null AND fast.next != null:
        slow = slow.next
        fast = fast.next.next
        IF slow == fast:
            RETURN true  # cycle exists
    RETURN false
```

8. Top K Elements (Heap)

Use: Kth largest, smallest, frequent elements

```
FUNCTION topK(arr, k):
    CREATE minHeap
    FOR each element in arr:
        ADD element to minHeap
        IF heap size > k:
            REMOVE min element from heap
    RETURN elements in heap
```

9. Subset / Subsequence / Combination

Use: Power set, string/array subsets

```
FUNCTION generateSubsets(arr):
    result = []
    FUNCTION backtrack(start, path):
        ADD copy of path to result
        FOR i FROM start TO length(arr)-1:
            ADD arr[i] to path
            backtrack(i+1, path)
            REMOVE arr[i] from path
    CALL backtrack(0, [])
    RETURN result
```

10. Topological Sort

Use: DAG, course schedule, dependency resolution

```
FUNCTION topoSort(graph):
    visited = set()
    stack = empty
    FUNCTION dfs(node):
        MARK node visited
        FOR neighbor in graph[node]:
            IF neighbor not visited:
                CALL dfs(neighbor)
        PUSH node to stack
    FOR node in graph:
        IF node not visited:
            CALL dfs(node)
    RETURN reversed stack
```

11. Modified Binary Search

Use: Search in rotated array, first/last occurrence, sqrt

```
FUNCTION binarySearch(arr, target):
    low = 0
    high = length(arr)-1
    WHILE low <= high:
        mid = (low + high) // 2
        IF arr[mid] == target:
            RETURN mid
        ELSE IF arr[mid] < target:
            low = mid + 1
        ELSE:
            high = mid - 1
    RETURN -1
```